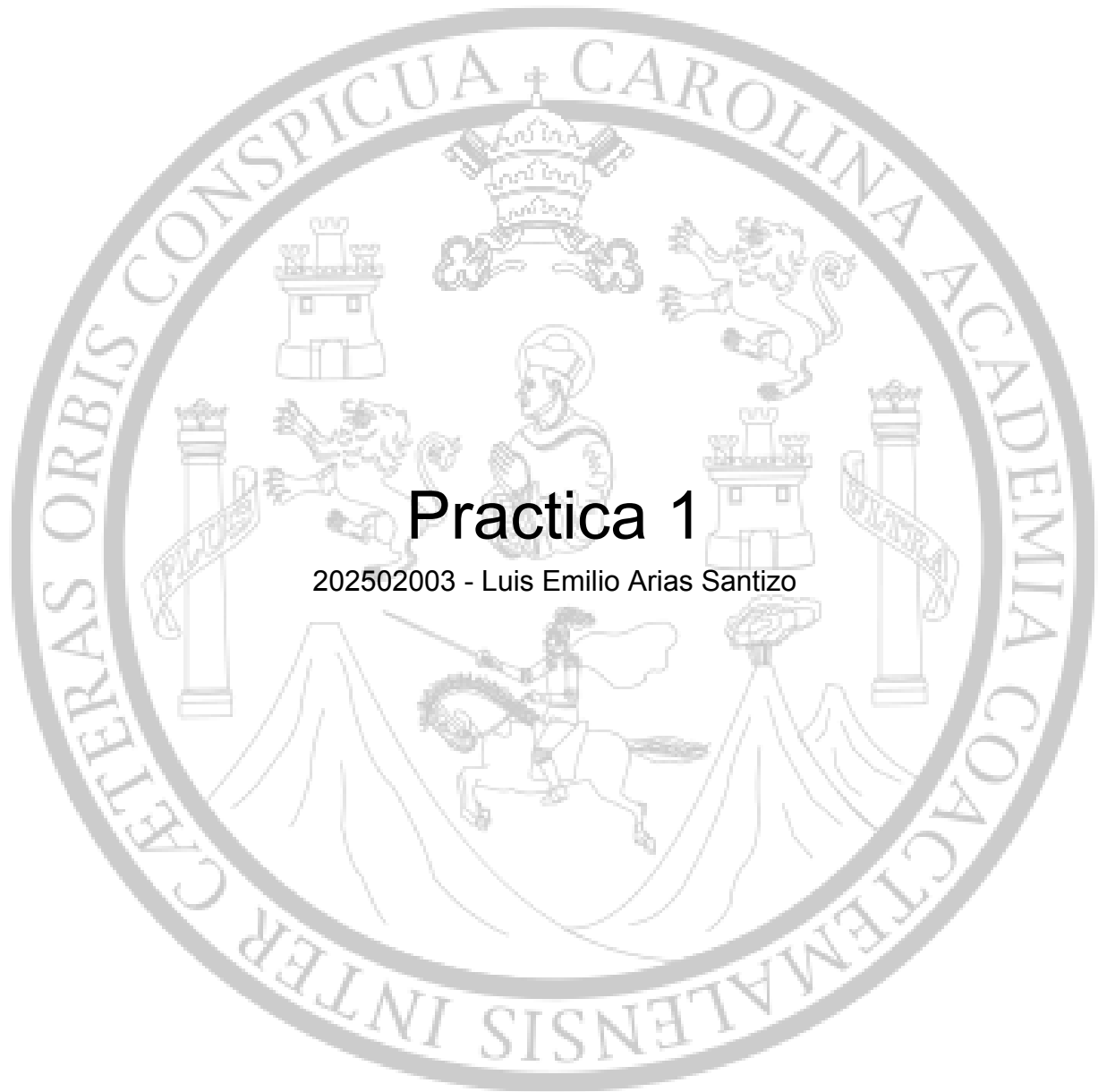


Universidad: San Carlos de Guatemala

Curso: Arquitectura de Computadores y Ensambladores 1

Sección: B



Practica 1

202502003 - Luis Emilio Arias Santizo

14 de septiembre del 2026

1. Introducción

La presente práctica consiste en el desarrollo de una calculadora de números enteros implementada completamente en lenguaje ensamblador ARM64 (AArch64). El programa funciona mediante una interfaz de consola que presenta un menú interactivo al usuario. A través de este menú es posible seleccionar diferentes operaciones aritméticas, ingresar los operandos correspondientes y visualizar el resultado obtenido.

La implementación permite aplicar conceptos fundamentales de programación a bajo nivel, entre ellos el uso de registros, instrucciones aritméticas, comparaciones, saltos condicionales, ciclos, subrutinas y llamadas al sistema de Linux para realizar operaciones de entrada y salida.

Además, se implementaron diferentes validaciones para evitar operaciones no permitidas, como la división entre cero, el uso de exponentes negativos y el cálculo del factorial de números negativos.

2. Objetivos

2.1 Objetivo general

Desarrollar una calculadora de números enteros utilizando lenguaje ensamblador ARM64, implementando operaciones aritméticas, estructuras de control, ciclos, validaciones y subrutinas.

2.2 Objetivos específicos

- Implementar las operaciones de suma, resta, multiplicación, división entera, potencia y factorial.
- Implementar un menú interactivo que permita ejecutar varias operaciones durante una misma sesión.
- Utilizar instrucciones aritméticas propias de la arquitectura ARM64.
- Implementar la potencia mediante multiplicación repetida.
- Implementar el factorial mediante un ciclo iterativo.
- Validar operaciones que no son permitidas por el programa.
- Utilizar subrutinas para organizar el funcionamiento de la aplicación.
- Utilizar GDB para realizar una inspección del programa desarrollado.

3. Herramientas utilizadas

Para el desarrollo y ejecución de la práctica se utilizaron las siguientes herramientas:

- Visual Studio Code.
- Ubuntu mediante WSL.
- Lenguaje ensamblador ARM64 / AArch64.
- GNU Assembler.
- GNU Linker.
- QEMU para ejecutar código ARM64.
- GDB Multiarch para la depuración del programa.

- Git y GitHub para el control de versiones y almacenamiento del proyecto.

El código fue desarrollado en archivos con extensión .s y posteriormente ensamblado y enlazado para obtener un ejecutable ARM64.

4. Descripción general del programa

El programa presenta inicialmente un menú con las siguientes opciones:

1. Suma.
2. Resta.
3. Multiplicación.
4. División entera.
5. Potencia.
6. Factorial.
7. Salir.

El usuario selecciona una opción mediante el teclado. Dependiendo de la operación seleccionada, el programa solicita uno o dos números enteros.

Para las operaciones de suma, resta, multiplicación, división y potencia se solicitan dos valores. Para el factorial únicamente se solicita un número.

Una vez realizada la operación, el resultado se convierte nuevamente a una representación de texto y se imprime en consola. Posteriormente, el programa regresa automáticamente al menú principal, permitiendo realizar una nueva operación sin tener que ejecutar nuevamente el programa.

La opción 7 permite finalizar la ejecución de manera controlada.

Evidencia del menú principal

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 7
laria@L2E0A2S5: /mnt/c/Proyectos_USAC/Arqui_Practica#1/ARQUI1B_2S_P_2025020003$
```

5. Operaciones implementadas

5.1 Suma

La operación de suma recibe dos números enteros ingresados por el usuario. Ambos operandos se almacenan en registros y posteriormente se utiliza la instrucción add de ARM64 para obtener el resultado.

Ejemplo:

$$10 + 5 = 15$$

La instrucción principal utilizada es:

```
add x20, x20, x21
```

Evidencia de suma

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 1
Ingrese el primer número: 10
Ingrese el segundo número: 5
Resultado: 15
```

5.2 Resta

La resta utiliza dos operandos ingresados por el usuario. Se emplea la instrucción sub para restar el segundo valor del primero.

Ejemplo:

$$10 - 20 = -10$$

La instrucción principal utilizada es:

```
sub x20, x20, x21
```

El programa permite mostrar correctamente resultados negativos gracias a la rutina de conversión de entero a texto.

Evidencia de resta

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 2
Ingrese el primer número: 10
Ingrese el segundo número: 20
Resultado: -10
```

5.3 Multiplicación

Para la multiplicación se utiliza la instrucción mul de ARM64.

Ejemplo:

$$7 \times 6 = 42$$

La operación se realiza mediante:

mul x20, x20, x21

El resultado se almacena nuevamente en el registro utilizado posteriormente para imprimirlo.

Evidencia de multiplicación

```
laria@L2E0A255: /mnt/c/Proy  X  +  v
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 3
Ingrese el primer número: 7
Ingrese el segundo número: 6
Resultado: 42
```

5.4 División entera

La división utiliza la instrucción sdiv, permitiendo trabajar con valores enteros con signo.

La instrucción utilizada es:

sdiv x20, x20, x21

Antes de ejecutar la división se verifica que el divisor sea diferente de cero.

Ejemplo:

$$20 / 4 = 5$$

Evidencia de división

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 4
Ingrese el primer número: 20
Ingrese el segundo número: 4
Resultado: 5
```

5.5 Potencia

La potencia fue implementada utilizando un ciclo de multiplicaciones repetidas. Inicialmente se coloca el valor 1 en un registro acumulador. Posteriormente, la base se multiplica por dicho acumulador tantas veces como indique el exponente.

Por ejemplo, para:

$$2^5 = 32$$

Dentro del ciclo se utiliza la instrucción:

`mul x23, x23, x20`

y el exponente se disminuye mediante:

`sub x21, x21, #1`

De esta forma se cumple con la implementación de la potencia mediante multiplicación repetida.

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 5
Ingrese el primer número: 2
Ingrese el segundo número: 5
Resultado: 32
```

5.6 Factorial

El factorial fue implementado mediante un ciclo iterativo.

El resultado comienza con el valor 1 y posteriormente se multiplica por el número actual mientras este sea mayor que uno.

Por ejemplo:

$$5! = 120$$

Durante cada iteración se utiliza:

`mul x23, x23, x20, sub x20, x20, #1`

El ciclo termina cuando el valor alcanza 1.

```
laria@L2E0A255: /mnt/c/Proy x +
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 6
Ingrese un número: 5
Resultado: 120
```

6. Validaciones implementadas

El programa incorpora diferentes validaciones para controlar operaciones que no deben realizarse.

6.1 División entre cero

Antes de ejecutar la instrucción de división, el segundo operando se compara con cero:

```
cmp x21, #0
```

```
beq division_cero
```

Cuando el divisor es cero, la operación no se ejecuta y se muestra el mensaje:

Error: no se puede dividir entre cero

Posteriormente, el programa regresa al menú principal.

Evidencia

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 4
Ingrese el primer número: 10
Ingrese el segundo número: 0
Error: no se puede dividir entre cero
```

6.2 Exponente negativo

La operación de potencia únicamente permite exponentes enteros no negativos.

Para verificar esta condición se realiza:

```
cmp x21, #0
```

```
blt exponente_negativo
```

Si el exponente es negativo, se muestra:

Error: el exponente no puede ser negativo

Evidencia

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 5
Ingrese el primer número: 2
Ingrese el segundo número: -3
Error: el exponente no puede ser negativo
```

6.3 Factorial de un número negativo

El factorial solamente se encuentra definido dentro del programa para números enteros no negativos.

Antes de iniciar el ciclo se comprueba:

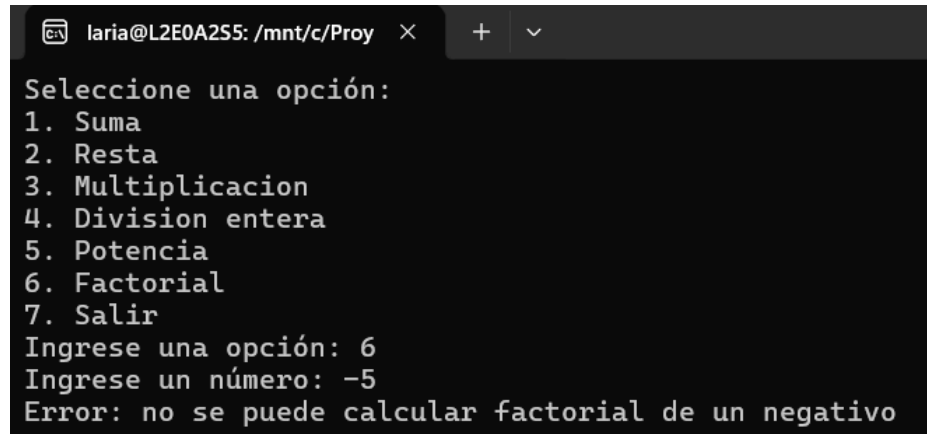
```
cmp x20, #0
```

```
blt factorial_negativo
```

En caso de recibir un número negativo se muestra:

Error: no se puede calcular factorial de un negativo

Evidencia



```
laria@L2E0A255: /mnt/c/Proy × + v
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 6
Ingrese un número: -5
Error: no se puede calcular factorial de un negativo
```

6.4 Opción inválida

El menú solamente acepta opciones comprendidas entre 1 y 7.

Cuando el usuario introduce otro valor, por ejemplo:

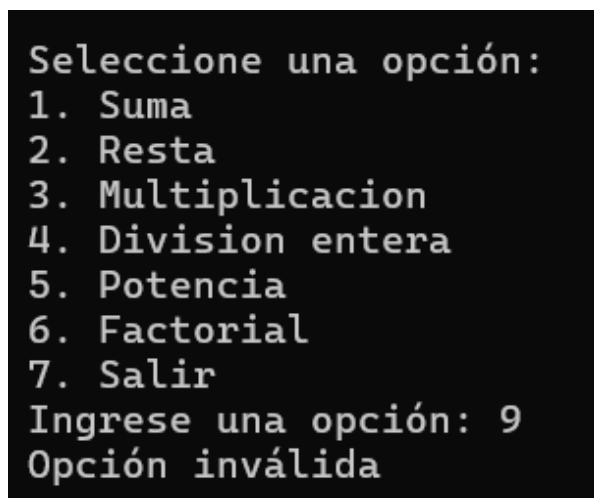
9

el programa muestra:

Opción inválida

y vuelve a presentar el menú.

Evidencia



```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 9
Opción inválida
```

7. Casos especiales

7.1 Potencia con exponente cero

Se implementó correctamente el caso en el cual el exponente es cero.

Por definición utilizada en el programa:

$$a^0 = 1$$

Por ejemplo:

$$2^0 = 1$$

El acumulador de potencia comienza con 1, por lo que cuando el exponente es cero el ciclo finaliza inmediatamente conservando dicho valor.

Evidencia

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 5
Ingrese el primer número: 2
Ingrese el segundo número: 0
Resultado: 1
```

7.2 Factorial de cero

También se contempla correctamente el caso:

$$0! = 1$$

El acumulador del factorial comienza con 1 y el ciclo solamente continúa mientras el número sea mayor que uno.

Por esta razón, cuando el usuario ingresa cero el resultado permanece en 1.

Evidencia

```
Seleccione una opción:
1. Suma
2. Resta
3. Multiplicacion
4. Division entera
5. Potencia
6. Factorial
7. Salir
Ingrese una opción: 6
Ingrese un número: 0
Resultado: 1
```

8. Conversión de datos

Debido a que los datos ingresados desde consola son recibidos como caracteres ASCII, fue necesario implementar rutinas para realizar conversiones entre texto y números enteros.

8.1 Rutina atoi

La rutina atoi convierte los caracteres ingresados por el usuario a su representación numérica.

Cada dígito leído se convierte de ASCII a número y el resultado acumulado se multiplica por diez antes de agregar el siguiente dígito.

También se contempla el carácter - al inicio del valor, permitiendo trabajar con números negativos.

El resultado de la conversión se almacena en el registro x10.

8.2 Rutina itoa

La rutina itoa realiza el proceso contrario: recibe un número entero y genera su representación en caracteres ASCII.

Esta rutina permite imprimir en consola los resultados obtenidos por las diferentes operaciones.

También maneja resultados negativos agregando el carácter - cuando corresponde.

9. Uso de registros

Durante el desarrollo del programa se utilizaron distintos registros para almacenar temporalmente operandos, resultados y estados de las operaciones.

Entre los principales se encuentran:

Registro x20: utilizado principalmente para almacenar el primer operando y posteriormente el resultado de la operación.

Registro x21: utilizado para almacenar el segundo operando.

Registro x22: utilizado para identificar la operación seleccionada antes de procesar los operandos.

Registro x23: utilizado como acumulador durante las operaciones iterativas de potencia y factorial.

Esta distribución permite mantener organizada la información necesaria durante la ejecución del programa.

10. Entrada y salida

La entrada y salida del programa se realiza mediante llamadas al sistema de Linux. Para leer información desde la entrada estándar se utiliza la syscall read, identificada con el número 63 en AArch64.

```
mov x0, #0
```

```
ldr x1, =input_buffer
```

```
mov x2, #64
```

```
mov x8, #63
```

```
svc #0
```

Para imprimir información se utiliza la syscall write, identificada con el número 64.

```
mov x0, #1
```

```
mov x8, #64
```

```
svc #0
```

Finalmente, para terminar el programa se utiliza la syscall exit, con número 93.

```
mov x0, #0
```

```
mov x8, #93
```

```
svc #0
```

11. Compilación y ejecución

Para ejecutar el programa desde una computadora con arquitectura diferente a ARM64 se utilizó QEMU.

La compilación y ejecución se realizó mediante el archivo Makefile.qemu utilizando el siguiente comando:

```
make -f Makefile.qemu SRC=src/11_practica.s run
```

El proceso utiliza GNU Assembler para generar el archivo objeto, GNU Linker para generar el ejecutable y posteriormente QEMU para ejecutar el binario ARM64.

12. Depuración con GDB

Para verificar el código generado se utilizó GDB Multiarch.

Se cargó el ejecutable mediante:

```
gdb-multiarch build/src/11_practica
```

Posteriormente se configuró la arquitectura:

```
set architecture aarch64
```

Durante la sesión de depuración se inspeccionaron las rutinas correspondientes a potencia y factorial mediante:

```
disassemble do_potencia
```

```
disassemble factorial
```

También se creó un breakpoint en la rutina de potencia:

```
break do_potencia
```

GDB identificó correctamente la rutina y la línea correspondiente dentro del archivo src/11_practica.s.

Adicionalmente, se utilizaron comandos como:

```
info breakpoints
```

```
info address do_potencia
```

```
info address factorial
```

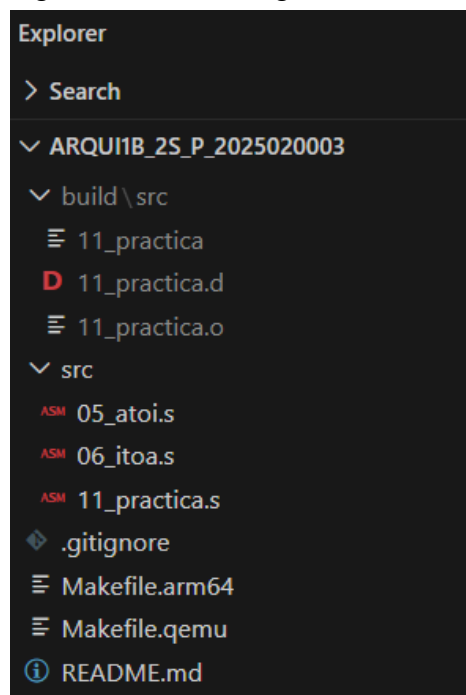
```
list do_potencia
```

Esto permitió verificar los símbolos generados, las direcciones de las rutinas, el breakpoint configurado y el código fuente asociado.

```
laria@L2E0A255: /mnt/c/Proy × + v
Reading symbols from build/src/11_practica...
(gdb) set architecture aarch64
The target architecture is set to "aarch64".
(gdb) disassemble do_potencia
Dump of assembler code for function do_potencia:
0x000000000040029c <+0>: cmp x21, #0x0
0x00000000004002a0 <+4>: b.lt 0x4002c4 <exponente_negativo> // b.tstop
0x00000000004002a4 <+8>: mov x23, #0x1 // #1
End of assembler dump.
(gdb) disassemble factorial
Dump of assembler code for function factorial:
0x00000000004002d4 <+0>: ldr x1, 0x4003f0 <exit+72>
0x00000000004002d8 <+4>: mov x2, #0x14 // #20
0x00000000004002dc <+8>: bl 0x400398 <print>
0x00000000004002e0 <+12>: bl 0x400368 <read>
0x00000000004002e4 <+16>: ldr x21, 0x4003c8 <exit+32>
0x00000000004002e8 <+20>: bl 0x4000b0 <atoi>
0x00000000004002ec <+24>: mov x20, x10
0x00000000004002f0 <+28>: cmp x20, #0x0
0x00000000004002f4 <+32>: b.lt 0x400318 <factorial_negativo> // b.tstop
0x00000000004002f8 <+36>: mov x23, #0x1 // #1
End of assembler dump.
(gdb) break do_potencia
Breakpoint 1 at 0x40029c: file src/11_practica.s, line 193.
(gdb) info breakpoints
Num Type Disp Enb Address What
1 breakpoint keep y 0x000000000040029c src/11_practica.s:193
(gdb) info address do_potencia
Symbol "do_potencia" is at 0x40029c in a file compiled without debugging.
(gdb) info address factorial
Symbol "factorial" is at 0x4002d4 in a file compiled without debugging.
(gdb) list do_potencia
188
189 b _start
190
191
192 do_potencia:
193 cmp x21, #0
194 blt exponente_negativo
195
196 mov x23, #1
197
(gdb) |
```

13. Estructura del proyecto

El proyecto se encuentra organizado de la siguiente manera:



El archivo 11_practica.s contiene la lógica principal de la calculadora.

El archivo 05_atoi.s contiene la rutina utilizada para convertir texto a números enteros y 06_itoa.s contiene la rutina utilizada para convertir números enteros a texto.

Los archivos Makefile permiten facilitar el proceso de ensamblado, enlazado y ejecución del programa.

14. Conclusiones

El desarrollo de la práctica permitió implementar una aplicación completa utilizando lenguaje ensamblador ARM64 y comprender de forma práctica el funcionamiento de instrucciones aritméticas, registros, comparaciones, saltos condicionales, ciclos y subrutinas.

La implementación de las operaciones de potencia y factorial permitió aplicar estructuras iterativas directamente en lenguaje ensamblador. De igual manera, las diferentes validaciones permitieron utilizar comparaciones y saltos condicionales para controlar el flujo del programa.

El uso de llamadas al sistema permitió realizar operaciones de entrada y salida directamente desde ensamblador, mientras que las rutinas de conversión hicieron posible transformar la información recibida desde consola a números enteros y posteriormente convertir los resultados nuevamente a texto.

Finalmente, el uso de GDB permitió inspeccionar las rutinas desarrolladas, establecer breakpoints y consultar símbolos y direcciones del programa, complementando el proceso de desarrollo y verificación de la calculadora ARM64.

15. Enlace al repositorio

Repositorio de GitHub:

https://github.com/larias-1302/ARQUI1B_2S_P_2025020003.git